Flow Matching : principes, fonctionnement et application à la répartition intelligente des éléments d'un réseau d'antennes circulaire multi-anneaux

Eya Hammami, Isra Abessy, Mayssam Hergli, Malek Soltany,
Senda Derouiche, Adem Gharbi, Mowafak Jendli

Institut National des Sciences Appliquées et de Technologie (INSAT) – Université de Carthage, Tunis
Encadrant : Pr. Naoufel Ismail

Module : Réalisation des fonctions électroniques par les techniques d'intelligence artificielle – 2026/2027

Résumé

La conception d'un réseau d'antennes répondant à un cahier des charges (gain du lobe principal, niveau du lobe secondaire, largeur du faisceau à mi-puissance, direction de pointage) est un *problème inverse* : il faut remonter des performances souhaitées vers une géométrie. Ce problème est mal posé, car plusieurs architectures distinctes peuvent produire des diagrammes quasi identiques. Un réseau de neurones à propagation avant (FFNN) entraîné par régression, approche de référence du cours, ne prédit qu'une seule solution moyenne, qui peut tomber entre deux solutions valides. Ce rapport présente le *Flow Matching*, une famille récente de modèles génératifs qui apprend, par une simple régression, un champ de vitesse transportant un bruit gaussien vers la distribution des données. Nous en expliquons le principe et le fonctionnement, puis proposons une méthodologie pour l'utiliser comme générateur conditionnel d'architectures de réseaux circulaires multi-anneaux : le modèle propose plusieurs répartitions candidates du nombre d'éléments par anneau, chacune est vérifiée par le calcul exact du facteur de réseau, et la meilleure est retenue. Nous détaillons enfin l'architecture logicielle et celle du réseau de neurones, en expliquant et en justifiant chacun de leurs composants.

Mots-clés : Flow Matching, modèles génératifs, transport optimal, conception inverse, réseau d'antennes circulaire, beamforming, facteur de réseau.

---

1 Introduction

Les réseaux d'antennes sont au cœur de nombreux systèmes modernes : réseaux de capteurs et objets connectés (IoT), radars militaires et automobiles exploitant le *beamforming*, imagerie médicale micro-ondes et communications par satellite multi-faisceaux. En pilotant l'amplitude et la phase de chaque élément, on oriente le lobe principal vers une direction désirée tout en contrôlant sa largeur et le niveau des lobes secondaires.

Les *réseaux circulaires concentriques* offrent un balayage électronique sur 360° avec une réponse relativement invariante. Leur conception pose une question combinatoire : *combien d'éléments placer sur chaque anneau* pour satisfaire un cahier des charges donné ? Le sujet du module [1] formalise cette question : à partir de quatre spécifications — gain du lobe principal, gain du lobe secondaire (SSL), largeur du lobe à -3 dB (HPBW) et angle de pointage θ_0 — il faut déterminer le nombre d'éléments $(N_1, \ldots, N_M)$ de chaque anneau.

L'approche de référence du cours entraîne un FFNN qui associe directement les spécifications à l'architecture, sur 50 000 exemples simulés. Elle suppose implicitement qu'à chaque cahier des charges correspond *une seule* architecture. Or la relation inverse est « un-à-plusieurs » : un régresseur entraîné par erreur quadratique converge vers la *moyenne* des solutions possibles, qui peut ne correspondre à aucune solution réelle.

Les modèles génératifs conditionnels répondent naturellement à ce problème : au lieu de prédire une valeur, ils apprennent la *distribution* des architectures compatibles avec la spécification et permettent d'en tirer plusieurs. Parmi eux, le *Flow Matching* (FM) [2, 3, 4] s'est imposé depuis 2023 par sa simplicité d'entraînement, sa stabilité et sa rapidité d'échantillonnage [11].

Ce rapport présente le Flow Matching et son fonctionnement (sections 2–4), analyse le problème direct du réseau multi-anneaux (section 5), puis décrit comment nous l'utiliserons pour la répartition intelligente des antennes (section 6) : architecture logicielle, architecture du réseau de neurones et entraînement, chaque choix étant expliqué et justifié.

2 Positionnement parmi les modèles génératifs

Un modèle génératif apprend à produire de nouveaux échantillons suivant une distribution de données connue seulement à travers des exemples. Plusieurs familles coexistent :

- **GAN** [12] : un générateur et un discriminateur s'affrontent ; échantillonnage rapide, mais entraînement instable et tendance à « oublier » une partie des solutions (effondrement de modes) ;
- **VAE** [13] : un réseau *encodeur* compresse chaque donnée en quelques nombres, un réseau *décodeur* la reconstruit ; stables, mais échantillons souvent peu précis ;
- **Flots normalisants** [14, 6] : transformations inversibles ; vraisemblance exacte, mais architecture contrainte et entraînement coûteux ;
- **Modèles de diffusion** [7, 8] : on bruite progressivement les données puis on apprend à inverser ce processus ; très haute qualité, mais des centaines d'étapes à la génération ;
- **Flow Matching** [2, 3, 4] : on apprend directement un champ de vitesse qui transporte le bruit vers les données, par une régression aussi stable que celle de la diffusion, avec des trajectoires plus droites donc moins d'étapes.

3 Le Flow Matching

3.1 Générer par transport

L'idée du FM est de transformer progressivement un bruit simple en donnée. On part d'un point $\boldsymbol{x}_0$ tiré d'une gaussienne $\mathcal{N}(\mathbf{0}, I)$ et on le déplace pendant un temps fictif $t \in [0,1]$ selon un champ de vitesse v_θ paramétré par un réseau de neurones :

$$\frac{\mathrm{d}\boldsymbol{x}_t}{\mathrm{d}t} = v_\theta(\boldsymbol{x}_t, t, \boldsymbol{c}). \tag{1}$$

Le point d'arrivée $\boldsymbol{x}_1$ est l'échantillon généré, et $\boldsymbol{c}$ est la condition (pour nous, le cahier des charges). Si le champ est bien appris, les points d'arrivée suivent la distribution des données. Tirer des bruits différents donne des échantillons différents : c'est ainsi que le modèle propose plusieurs solutions pour une même condition.

Les flots normalisants continus [6] utilisaient déjà (1), mais s'entraînaient par maximum de vraisemblance, ce qui impose de résoudre l'équation différentielle à chaque itération : c'est lent et coûteux. Le FM supprime cette contrainte.

3.2 Chemin d'interpolation

Pour savoir quelle vitesse apprendre, le FM fixe à l'avance le trajet de chaque particule. On associe à chaque donnée $\boldsymbol{x}_1$ un bruit $\boldsymbol{x}_0$ et on les relie par une ligne droite parcourue à vitesse constante [2, 3] :

$$\boldsymbol{x}_t = (1-t)\,\boldsymbol{x}_0 + t\,\boldsymbol{x}_1, \qquad \frac{\mathrm{d}\boldsymbol{x}_t}{\mathrm{d}t} = \boldsymbol{x}_1 - \boldsymbol{x}_0. \tag{2}$$

Ce chemin, dit de *transport optimal*, est le plus court possible entre le bruit et la donnée. Le modèle de diffusion correspond à un autre choix de chemin, courbé, ce qui explique qu'il demande plus d'étapes à la génération.

3.3 Objectif d'entraînement

On entraîne le réseau à prédire la vitesse de ce trajet droit, à partir de la seule position intermédiaire $\boldsymbol{x}_t$:

$$\mathscr{L}(\theta) = \mathbb{E}_{t,\boldsymbol{x}_0,(\boldsymbol{x}_1,\boldsymbol{c})} \left\| v_\theta(\boldsymbol{x}_t, t, \boldsymbol{c}) - (\boldsymbol{x}_1 - \boldsymbol{x}_0) \right\|^2, \tag{3}$$

avec t uniforme sur $[0,1]$, $\boldsymbol{x}_0$ gaussien et $(\boldsymbol{x}_1, \boldsymbol{c})$ tiré de la base. C'est une simple régression : aucune équation différentielle n'est résolue pendant l'entraînement.

Pourquoi cela fonctionne. Un même point $\boldsymbol{x}_t$ peut se trouver sur les trajets de plusieurs paires (bruit, donnée), chacune avec sa propre direction. La régression par moindres carrés apprend alors la *moyenne* de ces directions, c'est-à-dire la vitesse qui, globalement, transporte toute la distribution de bruit vers toute la distribution des données. Lipman *et al.* [2] ont démontré que minimiser (3) revient exactement à apprendre ce champ moyen, bien qu'on ne le calcule jamais explicitement. Les trajets individuels se croisent, mais le champ moyen, lui, sépare proprement les différents modes de la distribution (figure 1) : aucune solution n'est perdue.

3.4 Génération

Une fois le réseau entraîné, on tire un bruit $\boldsymbol{x}_0$ et on intègre (1) numériquement, par exemple par la méthode d'Euler en S pas de taille $h = 1/S$:

$$\boldsymbol{x}_{t+h} = \boldsymbol{x}_t + h\, v_\theta(\boldsymbol{x}_t, t, \boldsymbol{c}). \tag{4}$$

Les trajectoires étant presque droites, 10 à 50 pas suffisent, contre plusieurs centaines pour la diffusion. La méthode de Heun (ordre 2) améliore encore la précision à coût comparable.

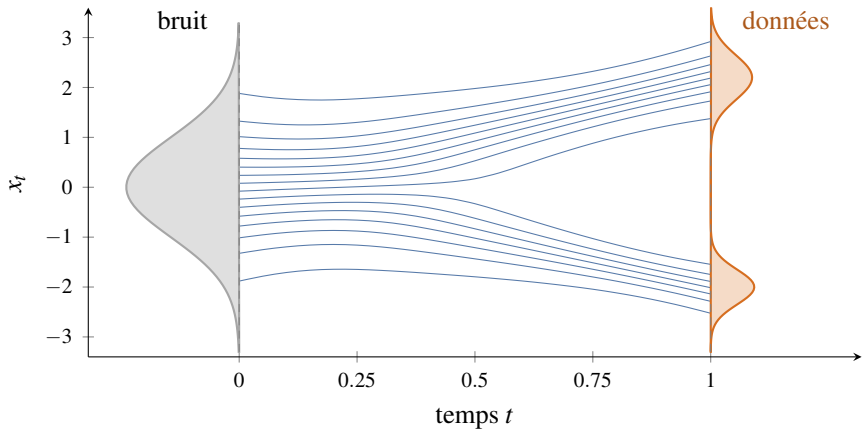


FIGURE 1 – Trajectoires du Flow Matching en dimension 1 : seize particules issues d'une gaussienne sont transportées vers une distribution cible à deux modes. Le champ de vitesse moyen est ici calculé exactement, puis intégré ; un réseau entraîné par (3) reproduit ce comportement. Les deux modes sont atteints.

3.5 Génération conditionnelle et guidage

Pour générer selon une condition $\boldsymbol{c}$, il suffit de la fournir en entrée du réseau et d'entraîner sur des paires (donnée, condition). Pour renforcer le respect de la condition, on utilise le *guidage sans classifieur* [9] :

pendant l'entraînement, la condition est masquée dans environ 10 % des cas, ce qui apprend au même réseau une version conditionnelle et une version non conditionnelle ; à la génération, on amplifie l'écart entre les deux avec un poids $w \geq 0$ qui règle le compromis entre fidélité et diversité.

3.6 Améliorations utiles

— **Couplage optimal en mini-lot** [5] : au lieu d'associer bruits et données au hasard, on les apparie au sein de chaque lot de façon à minimiser les distances parcourues. Les trajets se croisent moins, les trajectoires sont plus droites et l'apprentissage plus rapide.

— **Données discrètes** : le FM travaille sur des variables continues. Pour des entiers (nos nombres d'éléments), on peut soit les traiter comme des réels et arrondir à la fin, soit utiliser le *Discrete Flow Matching* [10], conçu pour les espaces finis. Nous commencerons par la première option, plus simple.

Les algorithmes 1 et 2 résument l'entraînement et la génération.

Algorithme 1 Entraînement du Flow Matching conditionnel

Entrée : base de paires $(\boldsymbol{x}_1, \boldsymbol{c})$, réseau v_θ

répéter
  tirer un lot $(\boldsymbol{x}_1, \boldsymbol{c})$, un bruit $\boldsymbol{x}_0 \sim \mathcal{N}(\mathbf{0}, I)$ et $t \sim \mathcal{U}[0,1]$
  (option) apparier $\boldsymbol{x}_0$ et $\boldsymbol{x}_1$ par transport optimal
  masquer $\boldsymbol{c}$ avec une probabilité de 10 %
  $\boldsymbol{x}_t \leftarrow (1-t)\boldsymbol{x}_0 + t\boldsymbol{x}_1$ ▷ éq. (2)
  descente de gradient (Adam) sur $\|v_\theta(\boldsymbol{x}_t, t, \boldsymbol{c}) - (\boldsymbol{x}_1 - \boldsymbol{x}_0)\|^2$
jusqu'à convergence

Algorithme 2 Génération de K candidats pour une condition $\boldsymbol{c}$

Entrée : condition $\boldsymbol{c}$, nombre de candidats K, nombre de pas S

tirer K bruits $\boldsymbol{x} \sim \mathcal{N}(\mathbf{0}, I)$; $h \leftarrow 1/S$
pour $s = 0, \ldots, S-1$ **faire**
  $\boldsymbol{x} \leftarrow \boldsymbol{x} + h\, v_\theta(\boldsymbol{x}, sh, \boldsymbol{c})$ ▷ éq. (4)
fin pour
retourner les K échantillons $\boldsymbol{x}$

4 Comparaison des approches

Le tableau 1 compare le FM aux autres approches vis-à-vis des besoins du projet : représenter plusieurs solutions, s'entraîner de façon stable et générer rapidement. Nos données étant de très faible dimension (cinq entiers), générer des dizaines de candidats avec le FM prend une fraction de seconde sur un processeur ordinaire.

TABLE 1 – Comparaison qualitative des approches pour la conception inverse

Approche	Multi-solutions	Entraînement	Pas de génération	Vraisemblance
FFNN (MSE)	non	stable	1	–
GAN	partiel	instable	1	non
VAE	oui	stable	1	borne
Diffusion	oui	stable	100–1000	approx.
Flow Matching	**oui**	**stable**	**10–50**	**exacte**

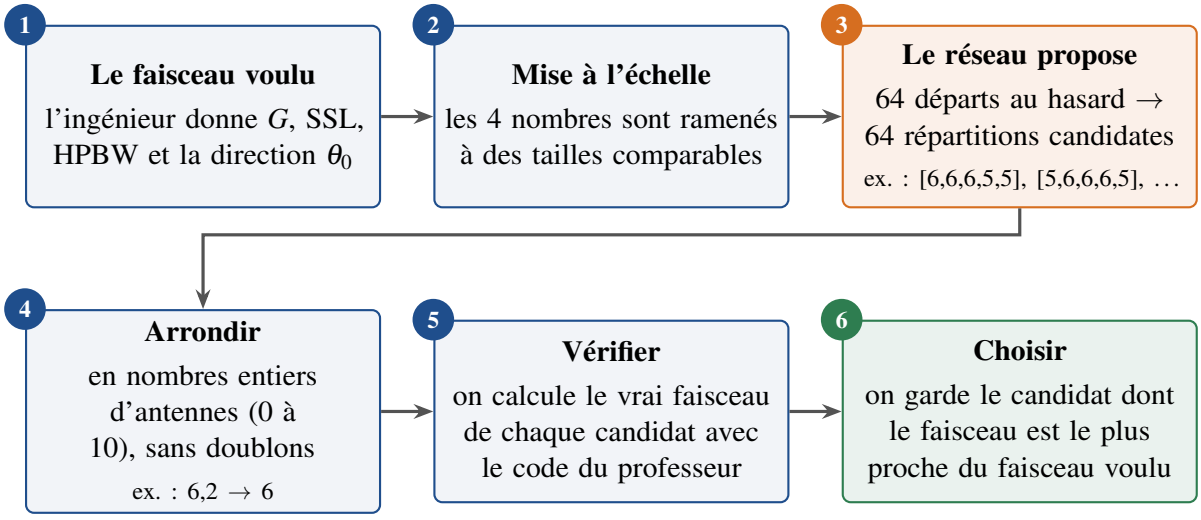


FIGURE 2 – Utilisation du modèle, étape par étape. Le réseau propose plusieurs réponses (3), la physique vérifie chacune d'elles (5) et on garde la meilleure (6) : aucune réponse n'est retenue sur la seule foi du réseau.

5 Le problème direct : réseau circulaire multi-anneaux

5.1 Facteur de réseau

Dans la configuration du projet [1], le réseau comporte au plus 5 anneaux concentriques dans le plan xy, de rayons $r_m = r_0 + (m-1)\Delta r$ avec $r_0 = 0{,}2\lambda$ et $\Delta r = 0{,}5\lambda$. L'anneau m porte $N_m \leq 10$ éléments isotropes d'amplitude unitaire, répartis uniformément aux angles $\phi_{mn} = 2\pi n/N_m$, à la fréquence $f = 2{,}45$ GHz ($\lambda \approx 12{,}24$ cm). Les phases d'excitation sont choisies pour orienter le lobe principal vers θ_0 [17]. Dans le plan de coupe $\phi = 0$, le facteur de réseau s'écrit alors

$$\mathrm{AF}(\theta) = \sum_{m=1}^{M} \sum_{n=1}^{N_m} e^{\,jkr_m \cos\phi_{mn}(\sin\theta - \sin\theta_0)}, \qquad k = \frac{2\pi}{\lambda}. \tag{5}$$

Du diagramme $20\log_{10}|\mathrm{AF}(\theta)|$ on extrait le gain du lobe principal G (maximum), le SSL (plus fort lobe secondaire, relatif à G) et la HPBW (largeur du lobe principal à -3 dB).

5.2 Deux propriétés structurelles

(P1) Le gain fixe le nombre total d'éléments. En $\theta = \theta_0$, tous les termes de (5) valent 1 et s'additionnent en phase, d'où

$$G = 20\log_{10} N_{\mathrm{tot}}, \qquad N_{\mathrm{tot}} = \textstyle\sum_m N_m. \tag{6}$$

Pour l'exemple de test du cours, $G = 28{,}94$ dB correspond à $N_{\mathrm{tot}} = 28$, ce que confirme l'architecture $[6,6,6,5,5]$. *Le gain ne renseigne donc que sur le total ; c'est la répartition entre anneaux qui façonne SSL et HPBW.* Or il existe 6 000 façons de répartir 28 éléments sur 5 anneaux avec 1 à 10 éléments chacun : le problème inverse est un problème de répartition fortement combinatoire, avec de nombreuses solutions voisines.

(P2) Symétrie $\theta_0 \leftrightarrow 180^\circ - \theta_0$. Le facteur de réseau ne dépend des angles qu'à travers $\sin\theta$ et $\sin\theta_0$. Deux lobes principaux apparaissent donc en θ_0 et $180^\circ - \theta_0$ (visibles dans les figures du cours en 45° et 135°) et fusionnent quand θ_0 est proche de 90°, d'où les grandes HPBW observées. θ_0 et $180^\circ - \theta_0$ décrivent ainsi *le même* cahier des charges : il est préférable de fournir $\sin\theta_0$ au modèle.

Nous avons recalculé le cas de test du cours avec (5) : pour $[6,6,6,5,5]$ et $\theta_0 = 112{,}36^\circ$, on obtient $G = 28{,}94$ dB, SSL $= -9{,}58$ dB et HPBW $\approx 76{,}4^\circ$, en accord avec le cours (28,94 dB, $-9{,}58$ dB, 76,76°). Ce calcul servira de simulateur de vérification (figure 3).

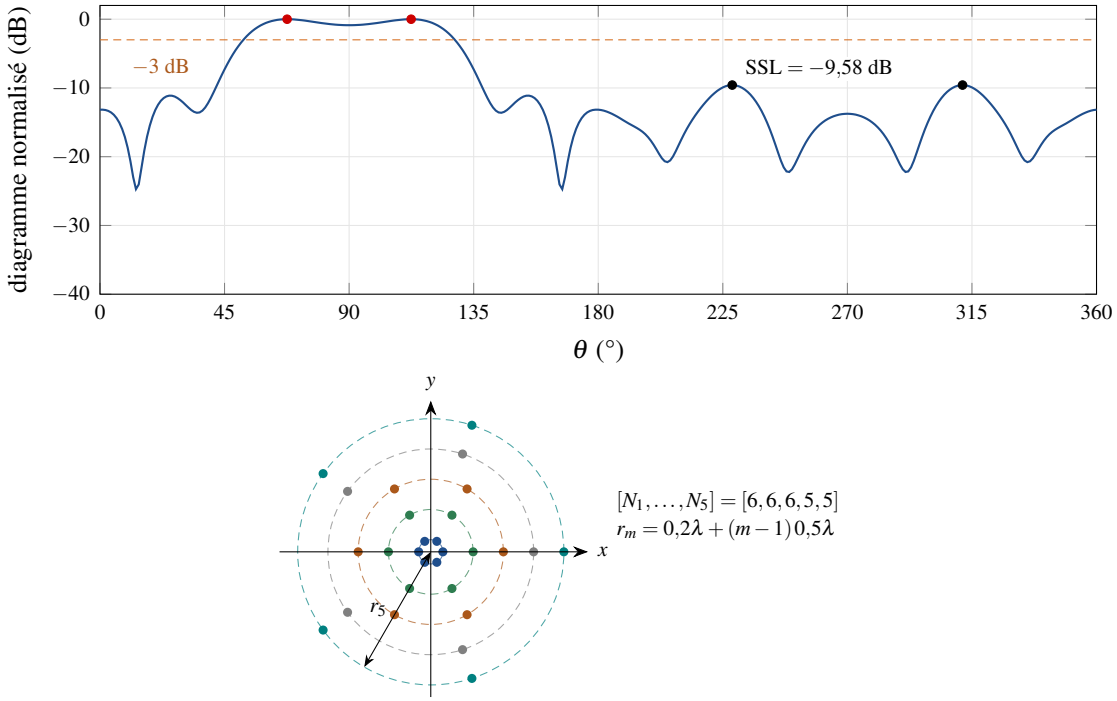


FIGURE 3 – Simulateur appliqué au cas de test du cours. En haut : diagramme normalisé pour $\theta_0 = 112{,}36°$; les deux lobes principaux en θ_0 et $180° - \theta_0$ (rouge) fusionnent en un faisceau de HPBW $\approx 76°$; le plus fort lobe secondaire (noir) est à $-9{,}58$ dB. En bas : géométrie correspondante (à l'échelle, $\lambda = 1$).

6 Application : répartition intelligente des antennes par Flow Matching

6.1 Formulation

La condition est le cahier des charges $\boldsymbol{c} = (G,\ \mathrm{SSL},\ \mathrm{HPBW},\ \theta_0)$ et la donnée à générer est l'architecture $\boldsymbol{x} = (N_1, \ldots, N_5)$, chaque N_m étant un entier entre 0 et 10 (la valeur 0 codant un anneau absent). Parmi plusieurs candidats, on retient celui dont les performances simulées s'écartent le moins du cahier des charges, mesuré par

$$\mathscr{J}(\boldsymbol{x}) = \sum_s w_s \, \frac{100 \left| s_{\text{simulé}}(\boldsymbol{x}) - s_{\text{demandé}} \right|}{S_s} \quad (\%), \tag{7}$$

avec $s \in \{G, \mathrm{SSL}, \mathrm{HPBW}\}$, des poids $w = (0{,}33\,;\, 0{,}33\,;\, 0{,}34)$ et des échelles $S_G = \max G$ et $S_{\mathrm{SSL}} = \max |\mathrm{SSL}|$ sur la base, $S_{\mathrm{HPBW}} = 180°$. Diviser par S_s exprime chaque écart en pourcentage de sa valeur maximale, ce qui rend comparables des décibels et des degrés. $\mathscr{J}$ est exactement l'*erreur globale pondérée* calculée par le code de test du professeur [1] : en l'utilisant comme note de sélection, nous optimisons directement le critère sur lequel le modèle est jugé. Elle est calculée par rapport au cahier des charges *demandé*, seule information disponible lors d'une utilisation réelle.

Pourquoi un FFNN ne suffit pas. Si deux architectures satisfont toutes deux la même spécification, un régresseur entraîné par erreur quadratique prédit leur moyenne, qui n'a aucune raison de la satisfaire — elle peut même ne pas être entière. La propriété (P1) montre que cette situation est la norme : toutes les répartitions d'un même total partagent le même gain. Le FM apprend au contraire l'ensemble des architectures compatibles, et chaque échantillon en est une.

6.2 Architecture logicielle

Le code est organisé en quatre scripts Python (PyTorch), exécutés dans l'ordre (figure 4). Chaque choix répond à une contrainte précise :

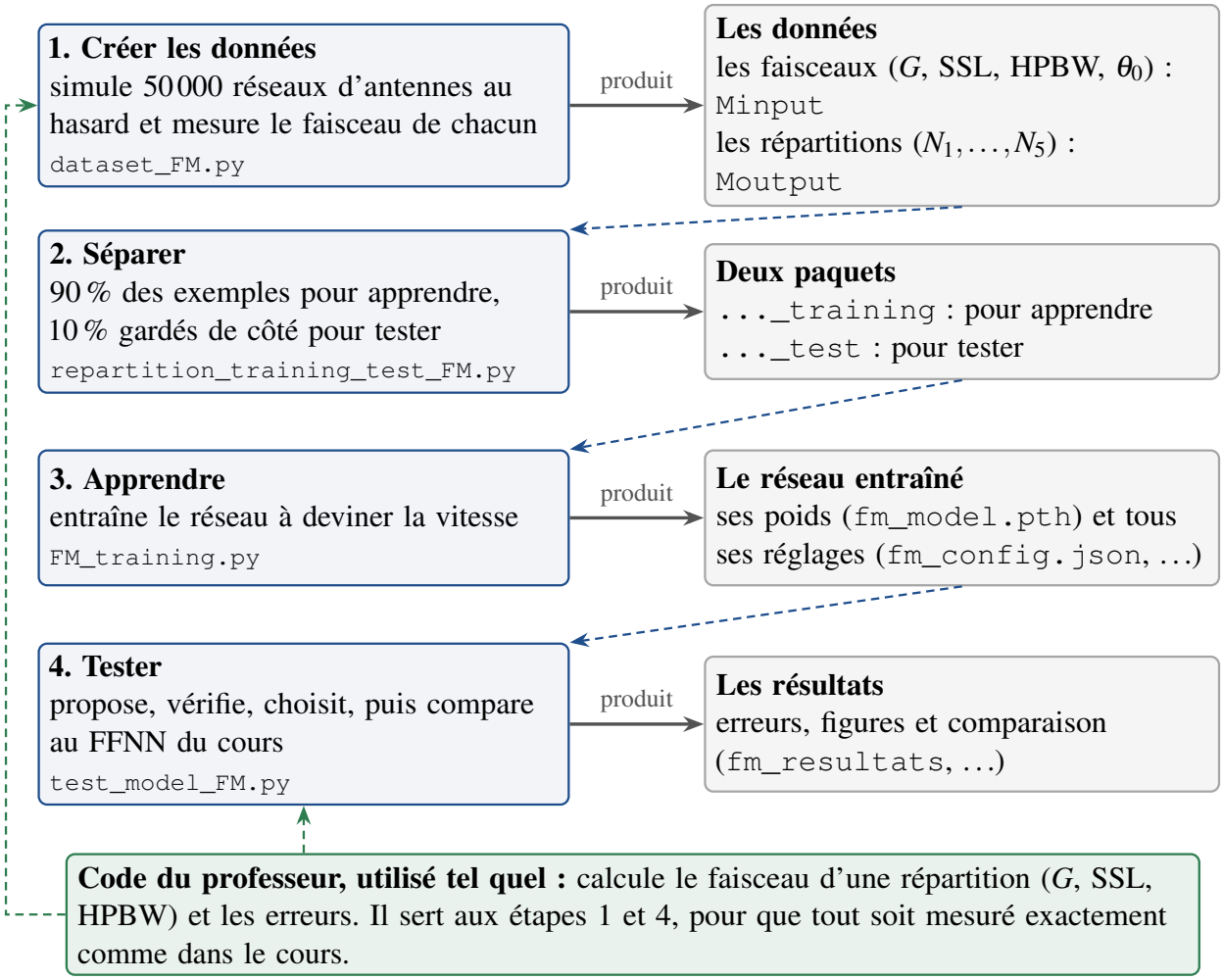


FIGURE 4 – Organisation du code : quatre étapes (à gauche), dont chacune produit des fichiers (à droite) ; les flèches pointillées bleues indiquent que ces fichiers sont utilisés par l'étape suivante. Les données sont enregistrées aux formats .npy (Python) et .mat (MATLAB).

- **Quatre scripts séparés** (exigence du professeur) : chacun fait une seule chose. On peut relancer une étape sans refaire les autres, par exemple réentraîner le réseau sans régénérer la base.
- **Communication uniquement par fichiers**, dont les noms, les dimensions et les unités sont fixés. Chaque script peut donc être écrit et vérifié seul. Les matrices suivent la convention du cours : une colonne par exemple, une ligne par grandeur.
- **Formats .npy et .mat** : .npy pour Python, .mat pour MATLAB (exigence du professeur).
- **Préfixe `fm_`** sur tous nos fichiers : ils n'écrasent jamais ceux du FFNN du cours, et les deux modèles peuvent tourner dans le même dossier pour être comparés.
- **Code du professeur copié à l'identique** dans les scripts 1 et 4 : G, SSL, HPBW et les erreurs sont mesurés exactement comme dans le cours. Comme dans ses propres codes, chaque script reste autonome (aucun module partagé) ; un petit programme de vérification contrôle que les copies restent identiques.
- **Un seul fichier de réglages**, `fm_config.json`, écrit à l'entraînement : le script de test reconstruit le réseau à partir de ce fichier, ce qui empêche l'entraînement et le test de diverger.
- **Graines aléatoires fixées** (46, comme dans le cours) : deux exécutions donnent les mêmes résultats.

Le code est versionné sur un dépôt Git ; un journal y consigne, à chaque étape, ce qui a été fait, les décisions prises et les résultats obtenus.

6.3 Base de données

Le script 1 reprend le générateur du cours [1] : 50 000 exemples, au plus 5 anneaux et 10 éléments par anneau, $\theta_0 \in [0°, 180°]$. Pour chaque exemple, une architecture et un angle sont tirés au hasard, le facteur de réseau est calculé par (5) et les spécifications sont extraites avec le code du professeur. Une option d'*équilibrage* tire d'abord le nombre total d'éléments, puis une répartition compatible, pour que petits et grands réseaux soient représentés de façon égale : sans elle, les totaux moyens (autour de 25) dominent la base. Le script 2 sépare la base comme dans le cours : 90 % pour l'entraînement, 10 % pour le test. Le script 3 met lui-même de côté 10 % des exemples d'entraînement pour la *validation*, c'est-à-dire pour choisir le meilleur modèle sans jamais toucher aux exemples de test.

6.4 Mise à l'échelle des données

Avant d'entrer dans le réseau, les nombres sont ramenés à des échelles comparables. Sans cette étape, une grandeur exprimée par de grands nombres (une HPBW jusqu'à 180°) écraserait les autres (un SSL de quelques décibels) et l'apprentissage serait lent et instable.

Architecture. Chaque N_m est ramené dans $[-1, 1]$ par $\tilde{x}_m = 2N_m/10 - 1$, la même échelle que le bruit de départ $\mathcal{N}(\mathbf{0}, I)$, ce qui garde les trajets courts. Pendant l'entraînement seulement, on ajoute un petit bruit uniforme entre $-0{,}1$ et $0{,}1$ (un demi-pas entre deux entiers) : on dit qu'on *déquantifie* la donnée. Sans ce bruit, le réseau ne verrait que 11 valeurs isolées par anneau, alors que le Flow Matching suppose des données continues. À la sortie, on fait le chemin inverse : retour à l'échelle d'origine, arrondi à l'entier le plus proche, limitation entre 0 et 10.

Condition. Le réseau reçoit $\boldsymbol{c}' = (G,\ \mathrm{SSL},\ \mathrm{HPBW},\ \sin\theta_0)$. Chaque composante est *centrée-réduite* : on lui soustrait sa moyenne et on la divise par son écart-type, calculés sur la base d'entraînement et enregistrés dans `fm_normalization_params.npz` pour être réutilisés à l'identique au test. On donne $\sin\theta_0$ plutôt que θ_0 à cause de la propriété (P2) : θ_0 et $180° - \theta_0$ donnent le même diagramme, le réseau n'a donc pas à le découvrir seul.

Remarque 1 (vocabulaire) *Le Flow Matching ne comporte ni « encodeur » ni « décodeur » au sens des VAE (un réseau qui compresse une donnée, un autre qui la reconstruit). Il n'y a qu'*un seul *réseau, v_θ, qui prédit une vitesse. Les opérations ci-dessus ne sont que des changements d'échelle fixes, sans apprentissage.*

6.5 Architecture du réseau de vitesse

Le seul réseau de neurones du projet est le champ de vitesse $v_\theta(\boldsymbol{x}_t, t, \boldsymbol{c}')$ de l'équation (1). La figure 5 le montre vu de l'extérieur, la figure 6 détaille un bloc de calcul, et le tableau 2 explique et justifie chaque élément.

Le réseau répond à une seule question : *depuis cette position, à ce moment, pour ce faisceau, dans quelle direction et à quelle vitesse faut-il bouger ?* Il fonctionne en quatre temps :

1. **Agrandir :** les 5 nombres de la position $\boldsymbol{x}_t$ sont transformés en 256 nombres, pour donner au réseau de la place pour calculer ;
2. **Résumer le moment et le faisceau :** l'instant t et le cahier des charges $\boldsymbol{c}'$ sont transformés en 128 nombres, appelés le *contexte* ;
3. **Corriger 4 fois :** 4 blocs de calcul affinent successivement le résultat, chacun réglé par le contexte. Une même position reçoit donc une vitesse différente selon le moment du trajet et le faisceau demandé — exactement ce qu'exige le Flow Matching conditionnel ;
4. **Réduire :** les 256 nombres sont ramenés à 5, la vitesse prédite $\hat{\boldsymbol{v}}$, une composante par anneau.

Ce réseau compte environ 8×10^5 paramètres (environ 3 Mo). C'est beaucoup plus que le FFNN du cours (environ 600 paramètres), mais il apprend une tâche plus riche : non pas une réponse par cahier des

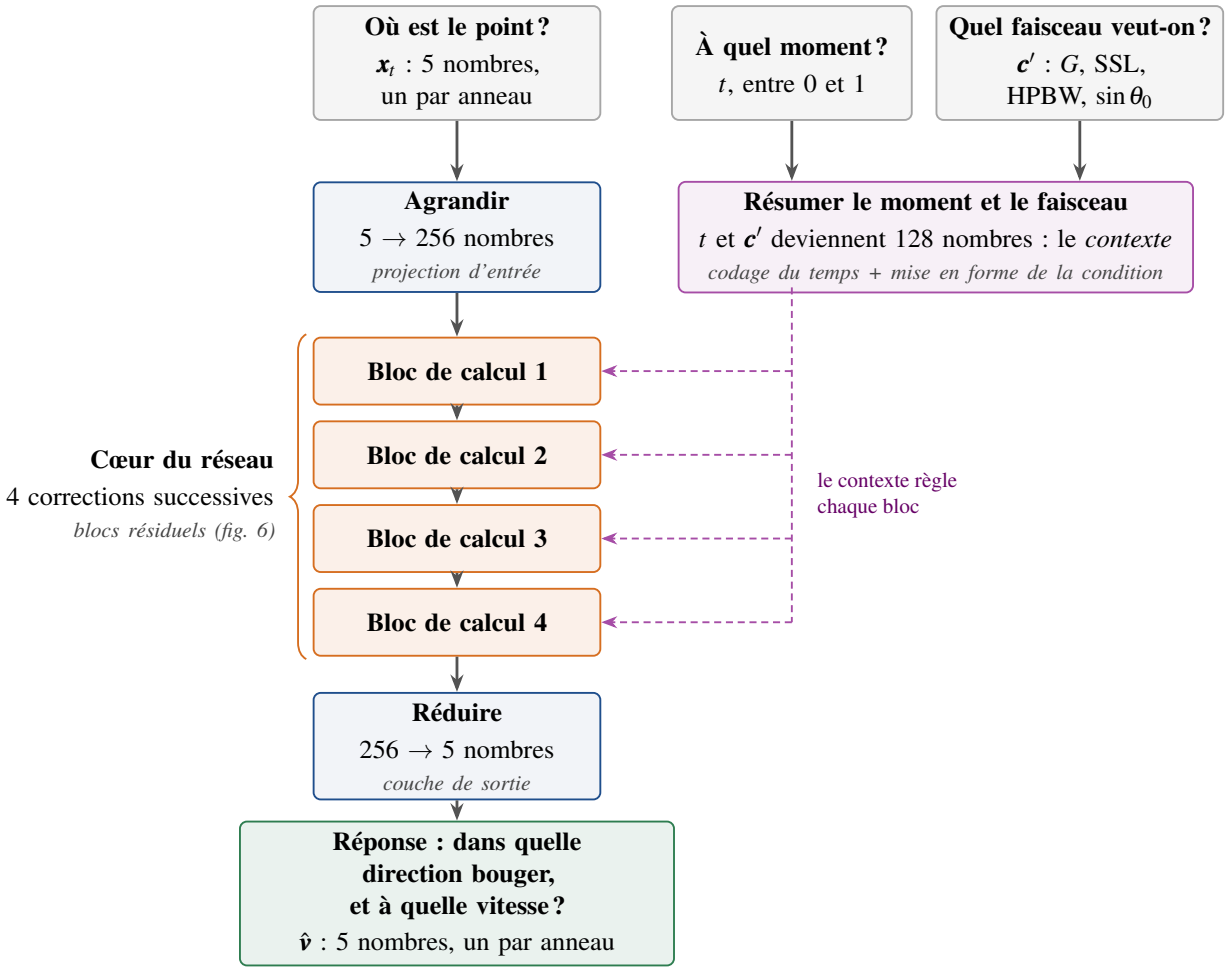


FIGURE 5 – Le réseau de neurones v_θ vu de l'extérieur. Il reçoit trois informations (en haut) et répond à une seule question (en bas) : depuis cette position, à ce moment, pour ce faisceau, dans quelle direction et à quelle vitesse faut-il bouger ? Les noms techniques sont en italique ; chaque élément est expliqué dans le tableau 2.

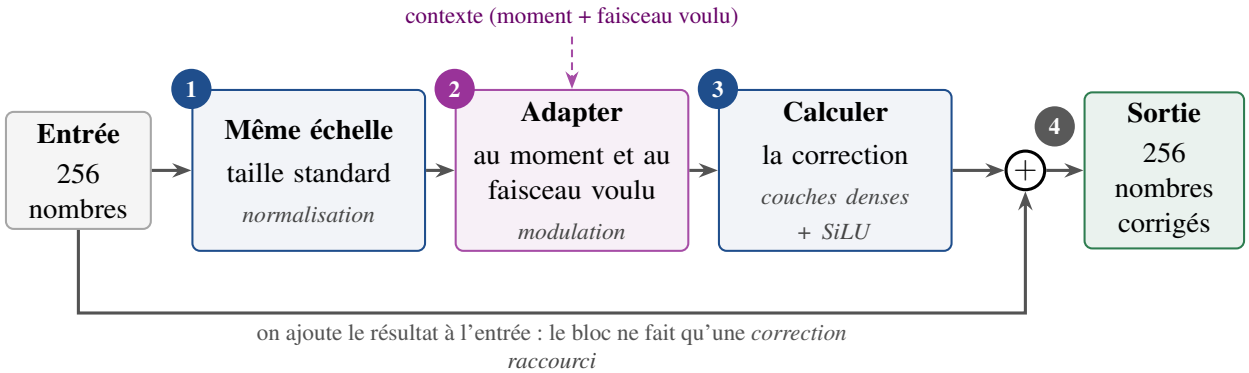


FIGURE 6 – Ce qui se passe dans un bloc de calcul. Le bloc ne remplace pas son entrée : il calcule une petite correction (étapes 1 à 3) puis l'ajoute à l'entrée (étape 4). Empiler 4 blocs revient donc à affiner le résultat 4 fois.

TABLE 2 – Composants du réseau de vitesse : ce qu'ils sont, ce qu'ils font, et pourquoi nous les avons choisis

Composant	Ce que c'est, en mots simples	Ce qu'il fait ici	Pourquoi ce choix
Les 3 entrées $\boldsymbol{x}_t$, t, $\boldsymbol{c}'$	La position actuelle du point (5 nombres, un par anneau), l'instant entre 0 et 1, le cahier des charges mis à l'échelle (4 nombres).	Ce sont les trois choses dont dépend la vitesse dans l'équation (1).	Rien de plus n'est nécessaire : le réseau ne connaît jamais la destination.
Couche de calcul *couche dense*	Chaque valeur de sortie est une somme pondérée de toutes les valeurs d'entrée, plus un décalage. Les *poids* sont les nombres ajustés par l'entraînement (`nn.Linear` en PyTorch).	Brique de base de tout le réseau, comme dans le FFNN du cours.	La plus simple et suffisante pour des données de 5 nombres : pas d'image (pas besoin de convolutions), pas de séquence (pas besoin de transformeur).
Agrandir *projection d'entrée* ($5 \to 256$)	Une couche dense qui transforme les 5 nombres de $\boldsymbol{x}_t$ en 256 nombres.	Donne au réseau un espace de travail plus large pour combiner les anneaux entre eux.	256 : assez pour représenter le champ de vitesse, assez peu pour s'entraîner sur un ordinateur portable.
Résumer le moment *codage du temps*	On remplace le nombre t par 64 valeurs $\sin(\omega_k t)$ et $\cos(\omega_k t)$, avec 32 fréquences ω_k allant du lent au rapide, puis une couche dense. Dans la littérature : *plongement sinusoïdal*.	Permet de distinguer finement deux instants proches ($t = 0{,}30$ et $t = 0{,}31$).	La vitesse change beaucoup au cours du trajet (hésitation puis décision, section 3) ; un seul nombre t est difficile à exploiter. Technique standard des modèles de diffusion et de FM.
Résumer le faisceau *mise en forme de la condition* ($4 \to 64 \to 64$)	Deux couches denses qui transforment les 4 spécifications en 64 nombres.	Combine les spécifications entre elles (par exemple G et HPBW) avant leur utilisation.	10 % du temps, le résultat est remplacé par un vecteur « vide » appris : le même réseau apprend ainsi aussi la version sans condition, nécessaire au guidage (section 3).
Contexte (128)	La mise bout à bout (*concaténation*) du codage du temps et de celui de la condition.	Résume « à quel moment du trajet » et « pour quel cahier des charges ».	Une seule information compacte envoyée à tous les blocs.
Bloc de calcul ($\times 4$) *bloc résiduel*	Un petit groupe de couches dont le résultat est *ajouté* à son entrée : $\boldsymbol{h}' = \boldsymbol{h} + F(\boldsymbol{h})$. Le bloc apprend une correction plutôt qu'une transformation complète.	Chaque bloc affine la représentation produite par le précédent.	Le *raccourci* laisse passer l'information et l'apprentissage directement, ce qui rend les réseaux de plusieurs blocs faciles à entraîner. 4 blocs : compromis entre précision et taille.
Même échelle *normalisation de couche*	Recentre et remet à l'échelle les 256 valeurs (moyenne 0, écart-type 1) à l'entrée de chaque bloc.	Garde les valeurs dans une plage raisonnable.	Évite que les valeurs explosent ou s'écrasent d'un bloc à l'autre : entraînement plus stable.
Adapter *modulation FiLM*	Chaque valeur est multipliée par un facteur γ puis décalée de β ; les 256 facteurs et 256 décalages sont calculés à partir du contexte par une couche dense. FiLM signifie *Feature-wise Linear Modulation* [15].	C'est ici que l'instant et le cahier des charges « pilotent » le calcul.	Plus efficace que de donner le contexte une seule fois à l'entrée : la condition agit dans chaque bloc, donc sur toute la profondeur du réseau.
Fonction non linéaire *activation SiLU*	Une fonction non linéaire, $\mathrm{SiLU}(u) = u \cdot \sigma(u)$ où σ est la sigmoïde : une version « douce » de la ReLU du FFNN du cours.	Sans elle, empiler des couches denses reviendrait à une seule opération linéaire.	Elle est lisse : la vitesse étant intégrée pas à pas (section 3), un champ lisse donne des trajectoires régulières. Standard en FM et en diffusion.
Réduire *couche de sortie* ($256 \to 5$)	Une normalisation puis une couche dense, sans activation.	Produit la vitesse prédite $\hat{\boldsymbol{v}}$, une composante par anneau.	Pas d'activation, car une vitesse peut être positive ou négative et de n'importe quelle taille.

charges, mais tout un champ de vitesse, défini en chaque point et à chaque instant. Il reste petit à l'échelle des réseaux actuels et s'entraîne sur le processeur d'un ordinateur portable.

6.6 Entraînement

La figure 7 détaille une itération de l'algorithme 1 : on tire un exemple, un bruit et un instant, on calcule le point correspondant sur la ligne droite, le réseau prédit une vitesse, on la compare à la vitesse de la ligne, et l'optimiseur corrige les poids. Le tableau 3 donne et justifie chaque réglage ; tous sont enregistrés dans

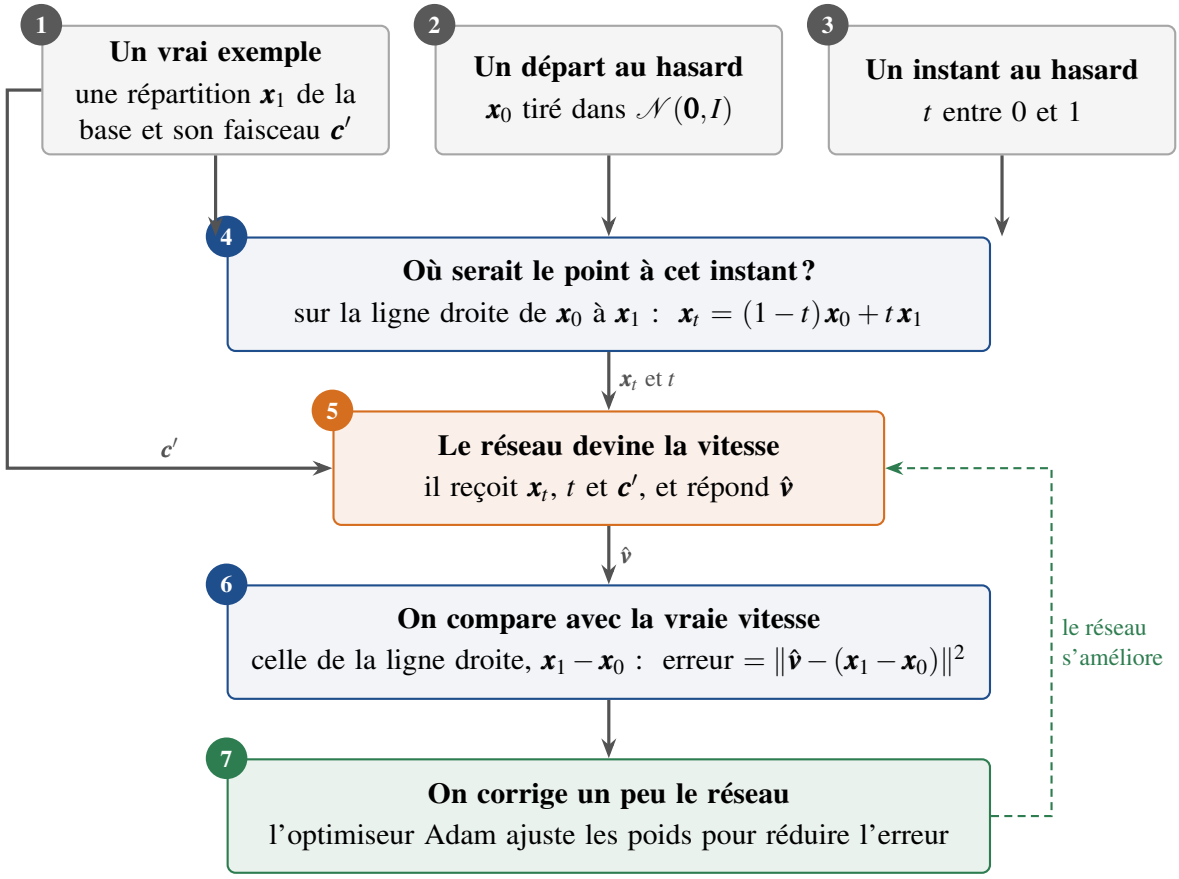


FIGURE 7 – Une étape d'entraînement. On recommence des dizaines de milliers de fois, par paquets de 256 exemples : à force de corrections, le réseau apprend à deviner la bonne vitesse partout.

`fm_config.json`.

TABLE 3 – Réglages de l'entraînement et justification

Réglage	**Valeur**	**Pourquoi**
Optimiseur	Adam [16]	Le même que pour le FFNN du cours : robuste et peu sensible au réglage.
Taux d'apprentissage	10^{-3}, décroissant	Taille des corrections de poids : grande au début pour avancer vite, de plus en plus petite pour affiner (décroissance en cosinus).
Lot	256 exemples	Nombre d'exemples traités ensemble : moyenne l'erreur sur plusieurs exemples, donc des corrections plus stables.
Époques	environ 200	Une époque = un passage sur toute la base. La perte du FM est bruitée (t et x_0 tirés au hasard) : il faut plus de passages qu'un FFNN.
Validation	10 % de l'entraînement	Exemples mis de côté pour choisir le meilleur modèle sans toucher au test.
Masquage de c'	10 %	Nécessaire au guidage sans classifieur (section 3).
Appariement	optimal, par lot	Associe chaque bruit à la donnée la plus proche : trajets plus droits (section 3).
Moyenne des poids	glissante, 0,999	On sauvegarde une moyenne des poids au fil de l'entraînement (*EMA*) plutôt que les derniers poids : elle lisse les fluctuations dues au hasard.
Déquantification	bruit $\pm 0{,}1$	Rend continues des données entières (section 6.4).
Graine	46	Comme dans le cours : résultats reproductibles.

6.7 Inférence : générer, vérifier, sélectionner

L'inférence exploite la nature générative du modèle (figure 2) :

1. **Proposer** $K = 64$ candidats pour le cahier des charges (algorithme 2 avec la méthode de Heun, 20 pas, guidage de poids w) ;
2. **Arrondir** : changement d'échelle inverse, arrondi, limitation entre 0 et 10, suppression des doublons et des architectures sans aucun élément ;
3. **Vérifier** : calculer le diagramme exact de chaque candidat avec le code du professeur ;
4. **Choisir** l'architecture qui minimise (7), et proposer aussi à l'ingénieur les meilleures solutions *distinctes* — par exemple celle qui utilise le moins d'éléments.

Aucune architecture n'est retenue sur la seule foi du réseau de neurones : le modèle propose, la physique vérifie. Une imperfection du générateur se traduit donc par quelques candidats rejetés, pas par une erreur de conception. En option, les candidats peuvent servir de population initiale aux métaheuristiques vues en cours (GA, PSO, GWO…), ou être affinés par une recherche locale (± 1 élément sur un anneau).

6.8 Protocole d'évaluation

Nous évaluerons le modèle sur la partie test avec les métriques du cours [1] : erreurs absolues et relatives sur G, SSL et HPBW, et erreur globale pondérée (7), calculées entre les spécifications demandées et celles *obtenues par simulation* de l'architecture retenue. Nous ne comparons pas l'architecture prédite à celle de la base : plusieurs architectures étant correctes, ce critère pénaliserait injustement une bonne solution différente. Nous ajoutons le *taux de validité* (proportion de cas où les trois écarts restent sous 0,5 dB, 1 dB et 3°), la *diversité* (nombre de solutions valides distinctes parmi les K candidats) et le *temps d'inférence*. Le tableau 4 résume le protocole.

TABLE 4 – Protocole expérimental prévu

Élément	**Choix**
Données	50 000 exemples ; 90 % entraînement, 10 % test (graine 46)
Référence 1	FFNN fourni en cours : 3 couches cachées de 14, 20 et 8 neurones, ReLU, Adam, taux 5×10^{-3}, lot 64, 100 époques
Référence 2	Algorithme génétique seul, même nombre de simulations
Proposé	Flow Matching + vérification + sélection ($K = 64$)
Variantes	avec/sans guidage ; avec/sans appariement optimal ; $K \in \{1, 8, 64\}$
Métriques	erreurs absolues et relatives (code du cours), erreur globale pondérée, MSE, MAPE ; validité ; diversité ; temps
Cas illustratif	$G = 28{,}94$ dB, SSL $= -9{,}37$ dB, HPBW $= 74{,}95°$, $\theta_0 = 112{,}36°$ (exemple du cours)

La variante $K = 1$ est particulièrement instructive : le FM s'y comporte comme un prédicteur à sortie unique, et l'écart avec $K = 64$ mesure directement ce qu'apporte la prise en compte des solutions multiples.

6.9 Extensions et plan de travail

Le FM ne faisant aucune hypothèse sur la nature des variables générées, le cadre s'étend naturellement : générer aussi des grandeurs continues (rayons des anneaux, amplitudes des éléments pour abaisser les lobes secondaires), utiliser le Discrete Flow Matching [10] pour supprimer l'arrondi, ou conditionner sur un gabarit de diagramme complet pour le beamforming multi-faisceaux.

Le développement suit six étapes, chacune validée et consignée dans le journal du dépôt avant de passer à la suivante : (0) mise en place du dépôt et exécution du FFNN du cours, qui sert de référence ; (1) scripts de base de données et de séparation, avec vérification du simulateur sur le cas du cours (section 5.2) ; (2) première version simple de l'entraînement ; (3) chaîne complète générer–vérifier–sélectionner et premières comparaisons avec le FFNN ; (4) ajout des améliorations une par une, chacune mesurée ; (5) version finale livrée.

7 Conclusion

Le Flow Matching apprend, par une simple régression, le champ de vitesse qui transporte un bruit gaussien vers la distribution des données ; le chemin en ligne droite rend l'entraînement stable et la génération rapide. Appliqué à la conception de réseaux d'antennes circulaires multi-anneaux, il répond à la nature « un-à-plusieurs » du problème inverse : le gain fixe le nombre total d'éléments, et c'est leur répartition entre anneaux, fortement combinatoire, que le modèle apprend. La chaîne proposée — génération de multiples candidats, vérification par le calcul exact du facteur de réseau, sélection — garantit que l'architecture retenue respecte réellement le cahier des charges, tout en offrant à l'ingénieur des solutions alternatives.

Références

[1] N. Ismail, « Réalisation des fonctions électroniques en utilisant les techniques d'intelligence artificielle : design d'un réseau d'antennes », support de cours, INSAT, Université de Carthage, 2026/2027.

[2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel et M. Le, « Flow matching for generative modeling », in *Proc. ICLR*, 2023.

[3] X. Liu, C. Gong et Q. Liu, « Flow straight and fast : Learning to generate and transfer data with rectified flow », in *Proc. ICLR*, 2023.

[4] M. S. Albergo et E. Vanden-Eijnden, « Building normalizing flows with stochastic interpolants », in *Proc. ICLR*, 2023.

[5] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf et Y. Bengio, « Improving and generalizing flow-based generative models with minibatch optimal transport », *Transactions on Machine Learning Research*, 2024.

[6] R. T. Q. Chen, Y. Rubanova, J. Bettencourt et D. Duvenaud, « Neural ordinary differential equations », in *Proc. NeurIPS*, 2018.

[7] J. Ho, A. Jain et P. Abbeel, « Denoising diffusion probabilistic models », in *Proc. NeurIPS*, 2020.

[8] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon et B. Poole, « Score-based generative modeling through stochastic differential equations », in *Proc. ICLR*, 2021.

[9] J. Ho et T. Salimans, « Classifier-free diffusion guidance », arXiv :2207.12598, 2022.

[10] I. Gat, T. Remez, N. Shaul, F. Kreuk, R. T. Q. Chen, G. Synnaeve, Y. Adi et Y. Lipman, « Discrete flow matching », in *Proc. NeurIPS*, 2024.

[11] P. Esser *et al.*, « Scaling rectified flow transformers for high-resolution image synthesis », in *Proc. ICML*, 2024.

[12] I. Goodfellow *et al.*, « Generative adversarial nets », in *Proc. NeurIPS*, 2014.

[13] D. P. Kingma et M. Welling, « Auto-encoding variational Bayes », in *Proc. ICLR*, 2014.

[14] D. J. Rezende et S. Mohamed, « Variational inference with normalizing flows », in *Proc. ICML*, 2015.

[15] E. Perez, F. Strub, H. de Vries, V. Dumoulin et A. Courville, « FiLM : Visual reasoning with a general conditioning layer », in *Proc. AAAI*, 2018.

[16] D. P. Kingma et J. Ba, « Adam : A method for stochastic optimization », in *Proc. ICLR*, 2015.

[17] C. A. Balanis, *Antenna Theory : Analysis and Design*, 4e éd. Hoboken, NJ, USA : Wiley, 2016.